SCS1309 Database Management

Lab Sheet - Stored Procedures, Functions & Triggers

UCSC is tracking student participation in practical sessions and their final examination performance in order to:

- Identify students who regularly attend practical sessions
- Analyze the relationship between attendance and exam results
- Detect students who may need academic support

As a **Database Administrator**, your task is to manage and analyze this data using SQL queries, stored procedures, and triggers.

Database Setup

A **database dump file** is provided with this lab.

Instructions:

1. Download the file: ucs_academic_tracking.sql
2. Import the database using the following command:

   SOURCE ucs_academic_tracking.sql;
3. Verify that all tables and sample data are available before starting the lab questions.

   Note: You are NOT required to create tables or insert data manually.

Part A – Stored Procedures & Functions

1) Write a stored procedure named GetStudentResults that accepts a *student_id* as input and displays:
- Student name
- Course name
- Exam marks
- Pass/Fail status

2) Modify the GetStudentResults procedure to display results only for courses where the student has attended 100% of the practical sessions (for each course).

3) Create a SQL function named AttendanceRate that accepts (*p_student_id, p_course_id*) and returns the attendance percentage for that student in that course.
- Return a value between 0 and 100
- Use *PracticalSessions* and *Attendance* to calculate the percentage

4) Write a stored procedure named GetAtRiskStudents that lists students who:
- Attended less than 50% of practical sessions (use your function or equivalent logic), and
- Failed the final exam

5) Write a stored procedure named GrantInstructorPrivileges that grants SELECT privileges on the ExamResults and Attendance tables to an instructor user.

6) Write a stored procedure named GetCourseAttendanceSummary that accepts (p_course_id) and returns, for each student in that course:
- student_id and student name
- total sessions for the course
- sessions_present, sessions_absent
- attendance_percentage

7) Create a SQL function named IsEligibleForExam that accepts (p_student_id, p_course_id) and returns:
- 1 if attendance ≥ 80%
- 0 otherwise

Part B – Triggers

The database dump already includes a summary table named StudentAttendanceSummary:

(student_id, total_present, total_absent, last_updated)

6. Create a BEFORE INSERT trigger on the ExamResults table that automatically sets the status column as follows:
    - Set status to 'Pass' if marks ≥ 50
    - Set status to 'Fail' otherwise
7. Create a BEFORE UPDATE trigger on the ExamResults table that automatically re-evaluates and updates the status column when marks are modified.

8. Create an AFTER UPDATE trigger on the ExamResults table that logs the old and new marks into the ExamResult_Audit table.
9. Create a BEFORE DELETE trigger that prevents deletion of an exam record if the student has passed the exam.
10. Create an AFTER INSERT trigger on the Attendance table that updates StudentAttendanceSummary:

- If status = 'Present', increase total_present
- If status = 'Absent', increase total_absent
- Create an AFTER DELETE trigger on the Attendance table that updates StudentAttendanceSummary:
- If status = 'Present', decrease total_present
- If status = 'Absent', decrease total_absent

Part C – Revision & Practice Questions

The following questions are based on topics covered in previous practical sessions. Attempt them for additional practice.

1) Retrieve students who enrolled before 2024 and belong to the School of Computing.
2) Find students who attended **all** the practical sessions.
3) Modify the query to show students who attended all sessions **and** passed the exam.
4) List all courses along with their credit values.
5) Find students who never attended a single practical session.
6) Find students who failed the exam but attended at least two practical sessions.
7) Using **ANY**, find students who scored higher than at least one student who passed.
8) Using **EXISTS**, find students who attended at least one session but failed the exam.
9) Create a view named **StudentPerformanceView** showing student name, number of sessions attended, and exam marks.
10) Query the view to list students who attended more than two sessions and passed the exam.
11) Create an index on the marks column and retrieve the **top two performers** in the Database Systems course.